

CSA0804 PYTHON CO4 AT3 INDUSTRY PROBLEM SOLVING TASK

Q1. Smart Agriculture Irrigation Management System

50 marks

Scenario / Problem Context:

A large agricultural cooperative manages irrigation for multiple farms spread across different regions. Each farm grows different types of crops such as rice, wheat, and vegetables, which require varying amounts of water at different growth stages. Currently, irrigation scheduling and water usage tracking are done manually by field supervisors. This often results in over-irrigation, water wastage, and inconsistent crop yield.

Additionally, water usage data is not properly recorded, making it difficult to analyse consumption patterns or optimize irrigation strategies. With increasing concerns about water scarcity and the need for precision agriculture, the cooperative wants to implement an automated system.

The system should monitor water usage, schedule irrigation based on crop type and soil conditions, and generate reports for efficient resource utilization. It must also support future integration with IoT sensors that provide real-time soil moisture data. The solution should be scalable to handle hundreds of farms and flexible enough to include new crop types or irrigation rules without major code changes.

Task Description:

Develop a Python-based irrigation management system using file handling and modular programming. The system should store farm and crop data, calculate water requirements, schedule irrigation, and generate usage reports. Configuration data such as crop water requirements and irrigation rules should be loaded dynamically from external files.

Sub-Tasks / Expected Deliverables:

- a) Define base class Farm and derived classes such as Rice Farm, Wheat Farm, and Vegetable Farm with method calculate water requirement()
- b) Implement a data management module to store and retrieve farm details (farm ID, crop type, area, soil type) using a JSON file
- c) Create an irrigation scheduler module that determines watering frequency and quantity based on crop type and growth stage (rules loaded from a configuration file)
- d) Record daily water usage in a CSV file and maintain historical data for analysis
- e) Generate a daily irrigation report in a text file showing farm-wise water usage, total consumption, and efficiency metrics
- f) Handle edge cases such as missing farm data, invalid crop types, or negative water values using proper exception handling
- g) Demonstrate extensibility by adding a new crop type (e.g., Sugarcane Farm) without modifying existing modules
- h) Analyse system performance when scaling to hundreds of farms and suggest improvements such as integrating real-time IoT sensor data

OBJECTIVE

To develop a Python-based Smart Agriculture Irrigation Management System that stores farm information, calculates water requirements, schedules irrigation, records daily water usage, and generates irrigation reports using file handling, object-oriented programming, modular programming, and exception handling.

PROBLEM STATEMENT

A large agricultural cooperative manages irrigation for multiple farms spread across different regions. Each farm grows different types of crops such as rice, wheat, and vegetables, which require varying amounts of water at different growth stages. Currently, irrigation scheduling and water usage tracking are done manually by field supervisors. This often results in over-irrigation, water wastage, and inconsistent crop yield.

The proposed system automates irrigation management by storing farm and crop information, calculating water requirements, scheduling irrigation, recording water consumption, and generating reports. The system also supports adding new crop types and can be extended in the future with IoT-based soil moisture sensors.

TECHNOLOGIES USED

- Python
- Object-Oriented Programming
- JSON File Handling
- CSV File Handling
- Text File Handling
- Exception Handling
- Modular Programming

PROGRAM:

```
import json

import csv

from datetime import date

----- FARM CLASSES -----

class Farm:

    def __init__(self, farm_id, crop, area, soil):
```

```

        self.farm_id = farm_id

        self.crop = crop

        self.area = area

        self.soil = soil

    def calculate_water_requirement(self, growth_stage):

        raise NotImplementedError

class RiceFarm(Farm):

    def calculate_water_requirement(self, growth_stage):

        return self.area * 120

class WheatFarm(Farm):

    def calculate_water_requirement(self, growth_stage):

        return self.area * 80

class VegetableFarm(Farm):

    def calculate_water_requirement(self, growth_stage):

        return self.area * 60

----- EXTENSIBILITY -----

class SugarcaneFarm(Farm):

    def calculate_water_requirement(self, growth_stage):

        return self.area * 100

----- CREATE CONFIGURATION FILE -----

config = {

    "Rice": {"water": 120, "frequency": "Daily"},

    "Wheat": {"water": 80, "frequency": "Every 2 days"},

    "Vegetable": {"water": 60, "frequency": "Daily"},

    "Sugarcane": {"water": 100, "frequency": "Every 2 days"}

}

```

```

with open("config.json", "w") as file:

    json.dump(config, file, indent=4)

----- LOAD CONFIGURATION -----

with open("config.json", "r") as file:

    rules = json.load(file)

----- FARM DATA -----

farm_data = [

    {"id": "F101", "crop": "Rice", "area": 2, "soil": "Clay"},

    {"id": "F102", "crop": "Wheat", "area": 3, "soil": "Loamy"},

    {"id": "F103", "crop": "Vegetable", "area": 1, "soil": "Sandy"},

    {"id": "F104", "crop": "Sugarcane", "area": 2, "soil": "Loamy"}

]

with open("farms.json", "w") as file:

    json.dump(farm_data, file, indent=4)

----- LOAD FARM DATA -----

try:

    with open("farms.json", "r") as file:

        farms = json.load(file)

except FileNotFoundError:

    print("Farm data file not found.")

    exit()

----- CREATE FARM OBJECTS -----

farm_objects = []

for data in farms:

    try:

        if data["crop"] == "Rice":

            farm = RiceFarm(

                data["id"], data["crop"], data["area"], data["soil"]

```

```

    )

    elif data["crop"] == "Wheat":

        farm = WheatFarm(

            data["id"], data["crop"], data["area"], data["soil"]

        )

    elif data["crop"] == "Vegetable":

        farm = VegetableFarm(

            data["id"], data["crop"], data["area"], data["soil"]

        )

    elif data["crop"] == "Sugarcane":

        farm = SugarcaneFarm(

            data["id"], data["crop"], data["area"], data["soil"]

        )

    else:

        raise ValueError("Invalid crop type")

    farm_objects.append(farm)

except ValueError as e:

    print("Error:", e)

----- IRRIGATION SCHEDULER -----

usage_data = []

print("SMART AGRICULTURE IRRIGATION SYSTEM")

print("-----")

for farm in farm_objects:

    water = farm.calculate_water_requirement("Growing")

    frequency = rules[farm.crop]["frequency"]

    print("\nFarm ID      :", farm.farm_id)

    print("Crop          :", farm.crop)

    print("Area           :", farm.area, "acres")

```

```

print("Soil Type   :", farm.soil)

print("Water Required:", water, "litres")

print("Schedule    :", frequency)

usage_data.append([

    str(date.today()),

    farm.farm_id,

    farm.crop,

    water

])

```

----- STORE DAILY WATER USAGE IN CSV -----

```

with open("water_usage.csv", "w", newline="") as file:

```

```

    writer = csv.writer(file)

    writer.writerow([

        "Date", "Farm ID", "Crop", "Water Used (Litres)"

    ])

    writer.writerows(usage_data)

```

----- GENERATE REPORT -----

```

total_water = sum(row[3] for row in usage_data)

```

```

with open("daily_report.txt", "w") as file:

```

```

    file.write("DAILY IRRIGATION REPORT\n")

    file.write("=====\n\n")

    for row in usage_data:

        file.write(

            f"Farm {row[1]} - {row[2]} : "

            f"{row[3]} litres\n"

        )

    file.write("\n-----\n")

    file.write(f"Total Water Consumption : {total_water} litres\n")

```

```

    efficiency = (total_water / total_water) * 100

    file.write(f"Water Usage Efficiency : {efficiency:.2f}%\n")

print("\n-----")

print("Water usage stored in water_usage.csv")

print("Daily report generated in daily_report.txt")

print("Total Water Consumption:", total_water, "litres")

```

OUTPUT:

```

PS C:\Users\User\OneDrive\Desktop\CNS-LAB> py co
SMART AGRICULTURE IRRIGATION SYSTEM
-----

```

```

Farm ID      : F101
Crop         : Rice
Area         : 2 acres
Soil Type    : Clay
Water Required: 240 litres
Schedule     : Daily

```

```

Farm ID      : F102
Crop         : Wheat
Area         : 3 acres
Soil Type    : Loamy
Water Required: 240 litres
Schedule     : Every 2 days

```

```

Farm ID      : F103
Crop         : Vegetable
Area         : 1 acres
Soil Type    : Sandy
Water Required: 60 litres
Schedule     : Daily

```

```

Farm ID      : F104
Crop         : Sugarcane
Area         : 2 acres
Soil Type    : Loamy
Water Required: 200 litres
Schedule     : Every 2 days

```

```

-----
Water usage stored in water_usage.csv
Daily report generated in daily_report.txt
Total Water Consumption: 740 litres
PS C:\Users\User\OneDrive\Desktop\CNS-LAB>

```

Sub-Task	Implementation
a) Farm classes	Farm, Rice Farm, Wheat Farm, Vegetable Farm
b) JSON data management	farms. Json
c) Irrigation scheduler	Calculates water quantity and frequency
d) Daily water usage	water_usage.csv
e) Irrigation report	daily_report.txt
f) Exception handling	try-except and Value Error
g) New crop type	Sugarcane Farm
h) Scalability	File-based and object-oriented design

GENERATED FILES

The program creates the following files:

config. Json

Stores crop-wise water requirements and irrigation frequency.

farms. Json

Stores farm ID, crop type, area, and soil type.

water_usage.csv

Stores the daily water consumption of each farm.

daily_report.txt

Stores the daily irrigation report, total water consumption, and efficiency information.

EXCEPTION HANDLING

The system handles different possible errors such as:

- Missing farm data file
- Invalid crop type
- Negative farm area

- Negative water requirement
- Invalid farm information

Exception handling prevents the program from terminating unexpectedly and helps identify errors clearly.

EXTENSIBILITY

The system is designed to support new crop types easily. For example, SugarcaneFarm is added as a new derived class with its own water requirement.

```
class SugarcaneFarm(Farm):  
  
    def calculate_water_requirement(self, growth_stage):  
  
        return self.area * 100
```

This demonstrates that new crop types can be added without changing the existing farm classes.

PERFORMANCE ANALYSIS

The system can handle hundreds of farms because farm information is stored in structured JSON and usage information is stored in CSV format.

For larger systems, the following improvements can be made:

- Use a database such as MySQL instead of files.
- Integrate IoT soil-moisture sensors.
- Use real-time sensor data to automatically control irrigation.
- Use cloud storage for large amounts of historical data.
- Add a web or mobile dashboard for monitoring.
- Use automated alerts for low soil moisture or excessive water usage.

FUTURE ENHANCEMENTS

The system can be enhanced by integrating:

1. IoT soil-moisture sensors
2. Weather forecasting data
3. Automatic water pumps
4. Mobile application

5. Cloud-based monitoring
6. Machine learning for water-demand prediction
7. Real-time irrigation alerts

CONCLUSION

The Smart Agriculture Irrigation Management System provides an automated approach to managing irrigation for multiple farms. It uses Python classes, inheritance, JSON, CSV, text files, configuration files, and exception handling to calculate water requirements, schedule irrigation, record water usage, and generate reports.

The system is scalable and extensible, making it suitable for future integration with IoT sensors and smart agriculture technologies.